

Создание базы данных на основе СУБД SQLite делается очень просто. При подключении мы указываем файл базы данных, и, если его нет, файл базы данных с указанным названием будет создан.

```
1  import sqlite3
2
3  """
4  Если бд есть в указанном каталоге, то
5  установим подключение к существующей бд.
6  Если нет - создадим новую (пустую)
7  """
8  con = sqlite3.connect('data.db')
```

Чтобы выполнять запросы на добавление, изменение или удаление данных, мы также будем использовать объект курсор. И вот здесь некоторая оговорка.

В sqlite3 есть два способа выполнять SQL-запросы:

- через соединение (`connection.execute()`);
- через курсор (`cursor.execute()`).

И в большинстве простых случаев - курсор вообще не обязателен.

`cursor` — это низкоуровневый инструмент, когда нужно больше контроля.

В каких случаях его лучше использовать:

- есть цикл по `SELECT`;
- сложная логика;
- несколько `fetch`;
- `bulk` операции;
- архитектурный проект.

Во всех остальных случаях (учебные задачи, маленькие скрипты, быстрый прототип) будет достаточно `connection.execute()`. Python сам создаёт временный `cursor` внутри `connection.execute()`, выполняет запрос и возвращает его. То есть фактически происходит:

```
cur = con.cursor()
cur.execute(...)
```

но скрыто.

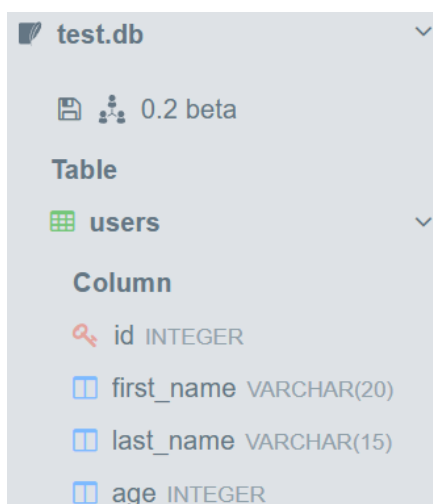
В крупных проектах, а также ряде библиотек (Django ORM, SQLAlchemy core), cursor остаётся базовой абстракцией DB-API и используется постоянно.

Вернёмся к запросам на добавление, изменение или удаление данных. В таких запросах нам уже не нужен fetch, а следовательно, не нужна и переменная для сохранения результата, ведь такие запросы ничего не возвращают.

Напишем программу для создания БД и таблицы в ней:

```
1  import sqlite3
2
3  con = sqlite3.connect('test.db')
4  con.execute('''
5      CREATE TABLE IF NOT EXISTS users (
6          id INTEGER PRIMARY KEY,
7          first_name VARCHAR(20),
8          last_name VARCHAR(15),
9          age INTEGER
10     )
11     ''')
12  con.close()
```

Проверяем созданную БД:



И убеждаемся в том, что всё выполнилось без ошибок. Теперь попробуем вставить в таблицу users первую запись:

```
con.execute('''
INSERT INTO users (first_name, last_name, age)
VALUES ('Ivan', 'Ivanov', 23)
''')
```

А также выполним запрос на получение всех пользователей, полный код:

```
1 import sqlite3
2
3 con = sqlite3.connect('test.db')
4 con.execute('''
5 INSERT INTO users (first_name, last_name, age)
6 VALUES ('Ivan', 'Ivanov', 23)
7 ''')
8 print(con.execute('SELECT * FROM users').fetchall())
9 con.close()
```

Получаем:

```
[(1, 'Ivan', 'Ivanov', 23)]
```

Видим, что всё успешно, пользователь якобы добавлен в таблицу. Но если попробовать вновь прочесть информацию из бд, после выполнения скрипта (можно даже на другой платформе), увидим, что таблица users пуста:



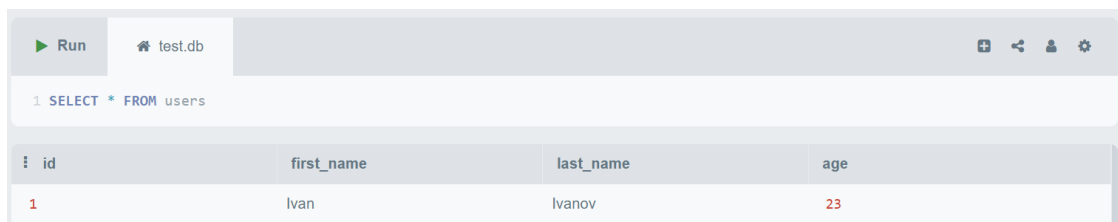
The screenshot shows a database client window with a tab labeled 'test.db'. The SQL editor contains the query '1 SELECT * FROM users'. Below the editor, a table structure is displayed with columns: 'id', 'first_name', 'last_name', and 'age'. The table body is empty, indicating no data was returned by the query.

id	first_name	last_name	age
----	------------	-----------	-----

По умолчанию все изменения с базой данных проводятся в памяти и не записываются в сам файл, поэтому если мы совершим манипуляцию с данными и перезапустим приложение, все правки пропадут. Чтобы такого не происходило, изменения надо зафиксировать. В этом нам поможет метод `commit()`, прописанный после внесения изменений (строка 8):

```
1  import sqlite3
2
3  con = sqlite3.connect('test.db')
4  con.execute('''
5  INSERT INTO users (first_name, last_name, age)
6  VALUES ('Ivan', 'Ivanov', 23)
7  ''')
8  con.commit()
9  con.close()
```

Проверяем:



id	first_name	last_name	age
1	Ivan	Ivanov	23

Вот теперь внесённые изменения сохранены.

Чтобы не держать в голове мысль, что нужно вовремя прописать `connection.commit()`, в python можно использовать контекстный менеджер, который используется при помощи оператора `with`.

Когда Python видит:

`with объект as имя:`

КОД

он автоматически вызывает у объекта два метода:

```
объект.__enter__() # вход в блок  
объект.__exit__() # выход из блока
```

То есть это просто автоматический «войти → поработать → корректно выйти». Соединение `sqlite3.connect()` — это объект, у которого уже реализован контекстный менеджер.

Если переписать нашу программу:

```
1  import sqlite3  
2  
3  with sqlite3.connect('test.db') as con:  
4      con.execute('''  
5          INSERT INTO users (first_name, last_name, age)  
6          VALUES ('Ivan', 'Ivanov', 23)  
7          ''')  
8  con.close()
```

Что происходит:

вход в блок `with`. Python делает: `con = connect(...)`, `con.__enter__()`.

Фактически мы просто получаем соединение;

работа с бд через `execute()`;

выход из блока `with`. Python делает: `con.__exit__(...)`. Если ошибок НЕ было, SQLite автоматически делает COMMIT. Если была ошибка, автоматически выполняется ROLLBACK, все изменения откатываются.

Важно знать, что `with` только управляет транзакцией (commit/rollback), но не гарантирует вызов `close()`.

Итак, `with` особенно полезен при изменении данных, но может использоваться и при SELECT, а также в таких случаях, когда:

маленький скрипт;

CLI-утилита;

учебный код;

соединение живёт недолго.

В долгоживущих приложениях часто создают собственный менеджер (с помощью класса или функции с try / except) и вызывать его с помощью with:

```
1  import sqlite3
2  from contextlib import contextmanager
3
4  @contextmanager 1 usage
5  def get_db(path="..."):
6      con = sqlite3.connect(path)
7      try:
8          yield con
9          con.commit()
10     except:
11         con.rollback()
12         raise
13     finally:
14         con.close()
15
16  with get_db() as con:
17      con.execute('...')
```

или

```
1  import sqlite3
2
3  class DB:
4      def __enter__(self):
5          self.con = sqlite3.connect("...")
6          return self.con
7
8      def __exit__(self, exc_type, exc, tb):
9          if exc_type:
10             self.con.rollback()
11         else:
12             self.con.commit()
13         self.con.close()
14
15  with DB() as con:
16      con.execute('...')
```